Software Engineering

System Analysis and Design





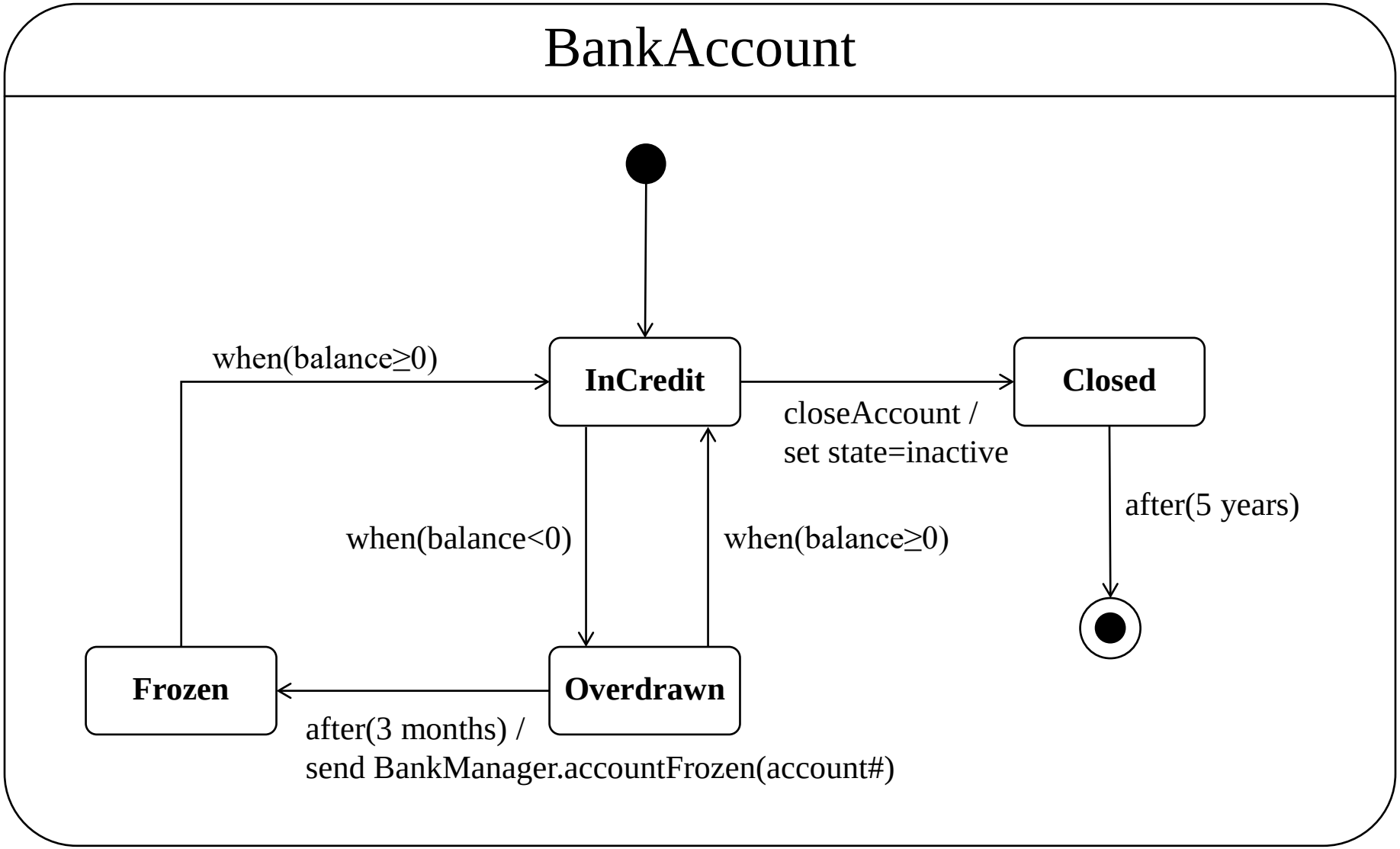


SYSTEM DESIGN OUTLINE

- ✓ System Design Overview
  - Life Cycle Role
  - The Purpose and Importance of System Design
  - Realizing Design Goals
  - Dealing with the Implementation Environment

- ➔ **System Analysis and Design Activities**
  - ✓ Architectural Analysis and Design
  - ✓ Use-case Analysis
  - ✓ Class Design
  - ☞ **Object Behaviour Analysis: State Machine Diagrams**
  - Design Patterns
  - Anti Patterns

EXAMPLE STATE MACHINE DIAGRAM



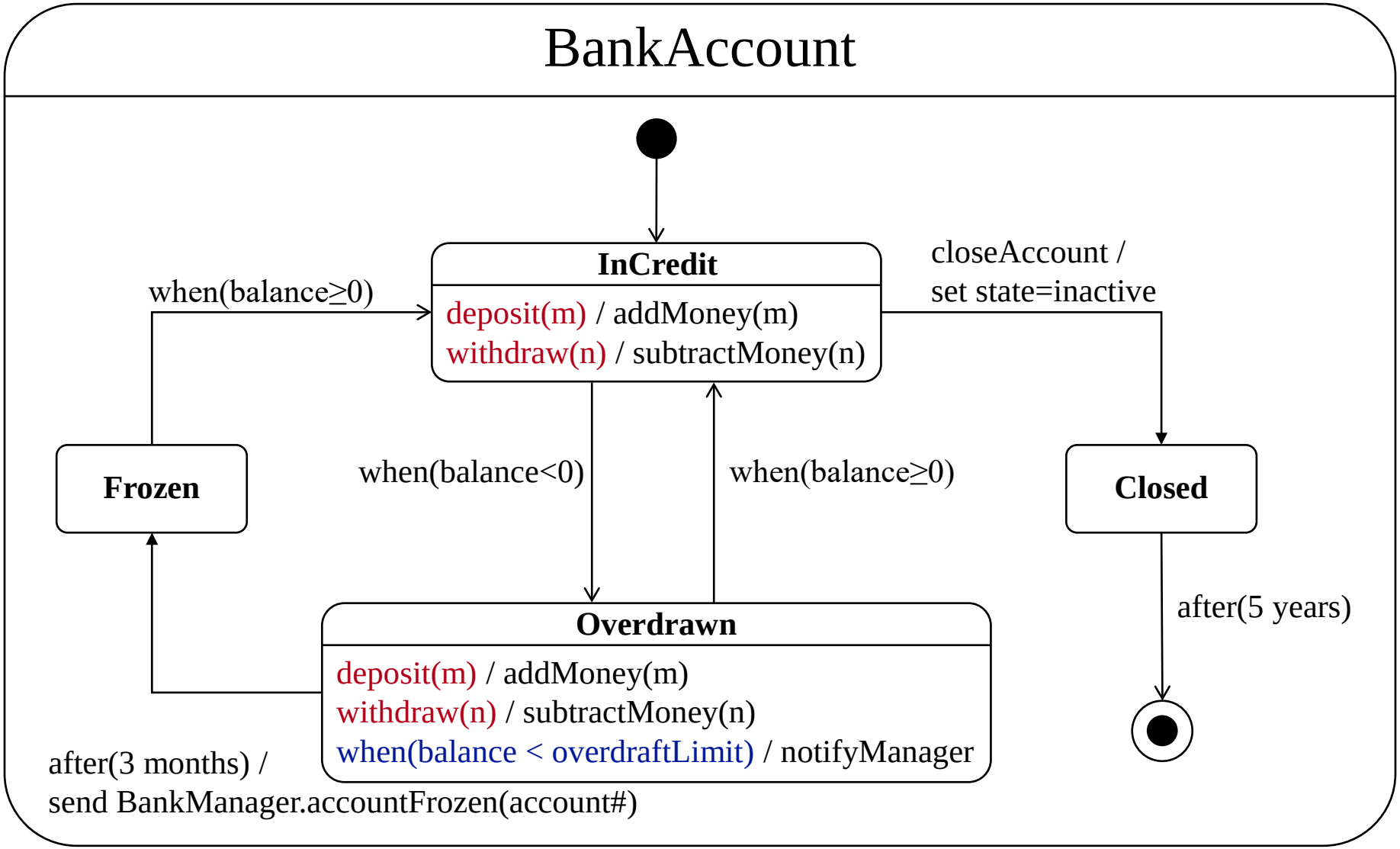
STATE MACHINE DIAGRAM

A *state machine diagram* describes the behavior inside an object.

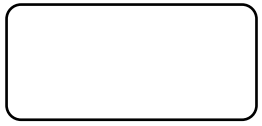
(What an object does when it receives a message.)

- It is a directed graph that shows:
  - the states of a single object (nodes).
  - the events that cause state changes (arcs).
- It shows all the messages that an object can send and receive.
- It describes all the possible states an object can get into during its life time.
- It is drawn for a single class to show the lifetime behavior of a single object.

EXAMPLE STATE MACHINE DIAGRAM



STATE MACHINE DIAGRAM: STATE



A *state* is a time during the life of an object when it satisfies some condition, performs some action or waits for an event.

☞ **A state has duration.**

- A state may be characterized by the:
  - value of one or more attributes of the class (e.g., a BankAccount object can be overdrawn or in credit based on the value of its *balance* attribute).
  - existence of a link to another object (e.g., a BankAccount object may be an individual or joint account based on the existence of links to one or two Customer objects).

STATE MACHINE DIAGRAM: SPECIAL STATES



initial state (start state)

☞ **Each diagram must have at most one initial state.**



final state (end state)

☞ **Each diagram can have multiple final states.**

☞ **No initial or final state indicates looping behaviour.**

- States may be named. 

- States may be unnamed (called anonymous states) 

STATE MACHINE DIAGRAM: TRANSITION

→ **A *transition* is a change of state from an originating state (source state) to a successor state (target state).**

☞ **The source and target states may be the same state.**

☞ **A transition takes zero time and cannot be interrupted.**

- Transition adornments (*all are optional*) include:

  event trigger [guard condition] / effect list

  - **event trigger** → an event name plus optional parameters.
    - An *event trigger* is the (implicit) event that causes a transition to occur.
    - The event is said to **trigger** the transition; the transition is said to **fire**.
  - **guard condition** → a Boolean expression which must be true for the transition to fire.
  - **effect list** → an atomic procedural expression executed if and when a transition fires.

STATE MACHINE DIAGRAM: EVENT

An *event* is something that happens at an instantaneous point in time.

Event types

call event – the receipt of a *synchronous call* from an object (a request that an operation be performed)

change event – a specified Boolean condition becoming true
→ when(balance < 0)

time event – absolute time → when(date=07/03/2009)
– elapsed time → after(10 seconds)

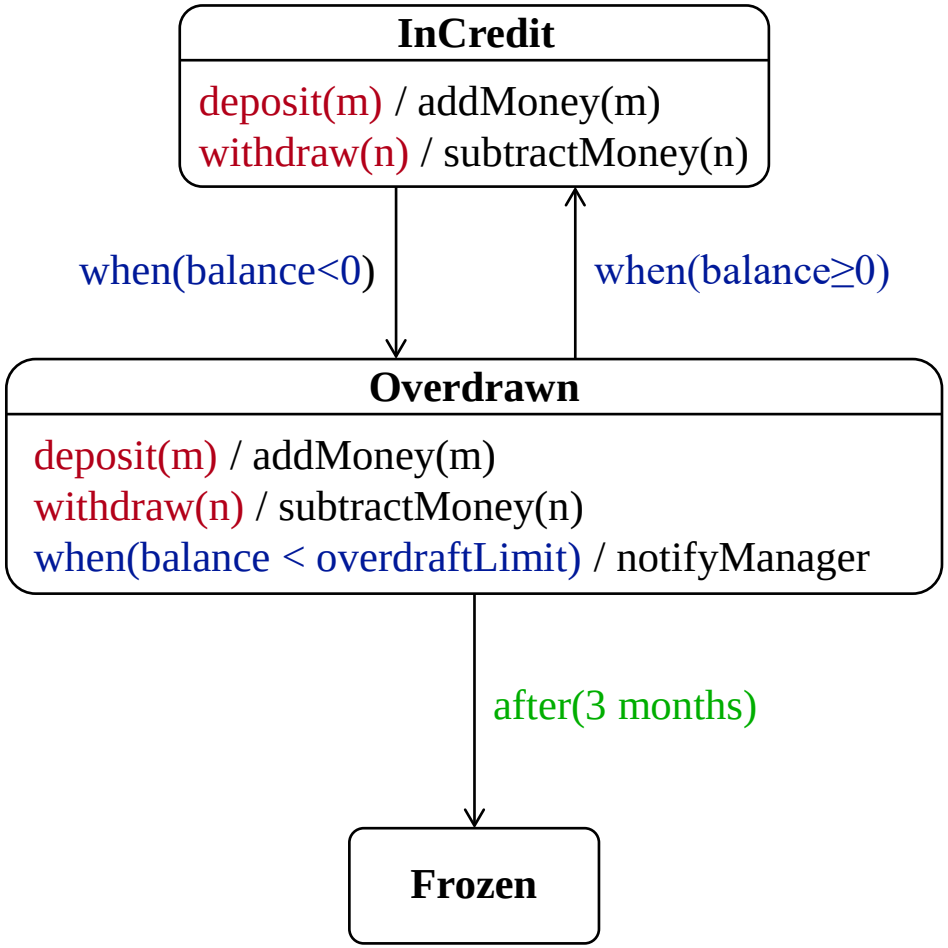
signal event – the receipt of an *asynchronous communication* from an object

STATE MACHINE DIAGRAM:
EVENT TYPE EXAMPLES

Call event

Change event

Time event



STATE MACHINE DIAGRAM: EVENT/TRANSITION MECHANICS

- Events are processed one at a time.

  ☞ **If the event does not trigger any transition it is ignored.**

- Only one transition within a state machine diagram may fire.
  - If two transitions can fire, then the choice may be nondeterministic (i.e., a race condition) → *This is probably a specification error!*

  ☞ **All transitions out of a state must correspond to different events.**

- There are two ways to transition out of a state:
  - **automatic** - when the activity of the state completes.

    ☞ **Transitions without labels fire immediately when the state activity, if any, completes.**

  - **non-automatic** - caused by an event.

STATE MACHINE DIAGRAM:
ACTIONS & ACTIVITIES

action - instantaneous and cannot be interrupted

☞ Processing for transitions between states and entry/exit of a state.

activity - takes time to complete and can be interrupted

☞ Processing that occurs while in a state.

- Possible forms of state behavior:

  no behavior → wait until an event occurs that exits the state.

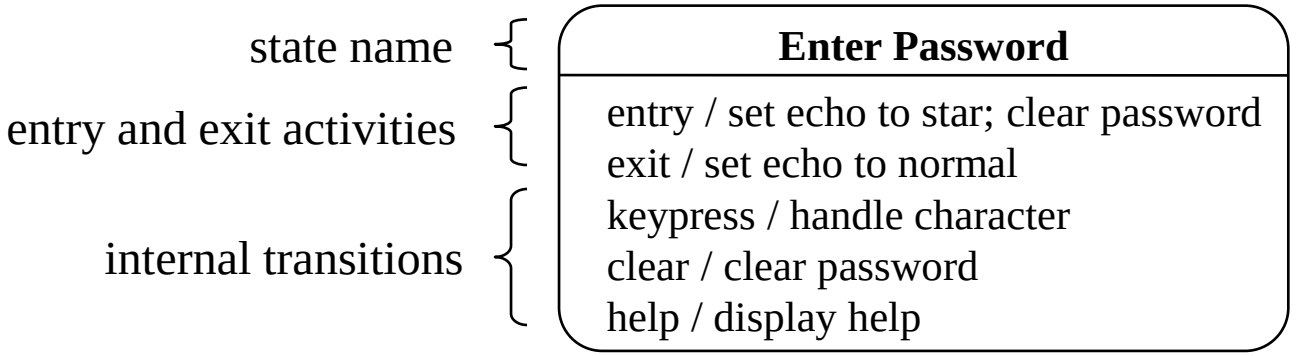
  event trigger [guard condition] / effect list → when *event trigger* occurs and *guard condition* is true, *effect list* is performed.

  do / activity → ongoing execution of behaviour (e.g., operations).

  entry / activity → performed every time the state is entered.

  exit / activity → performed every time the state is exited.

STATE MACHINE DIAGRAM:
ACTIONS & ACTIVITIES EXAMPLE



ASU COURSE REGISTRATION (REVISED)

At the beginning of each term, students may request a course catalogue containing a list of course offerings needed for the term. Information about each course, such as instructor, department, and prerequisites are included to help students make informed decisions.

The new system will allow students to select four course sections for the coming term. In addition, each student will indicate two alternative choices in case a course section becomes filled or is canceled. No course section will have more than forty students or fewer than ten students. A course section with fewer than ten students will be canceled. Once the registration process is completed for a student, the registration system sends information to the billing system so the student can be billed for the term.

Professors must be able to access the online system to indicate which courses they will be teaching, and to see which students signed up for their course offerings.

For each term, during the registration period students can change their schedule. Students must be able to access the system during this time to add or drop courses.

ASU STATE MACHINE DIAGRAM: SECTION CLASS

- To construct a state machine diagram we ask the following questions:
  - What states can the class be in?
  - What determines the state that the class is in?
  - To what events (messages) does each state respond and what happens when the event occurs?

The solution for the Section class state machine diagram is included in the solution to the lecture exercise.

COMPOSITE STATE MACHINE DIAGRAM

A *composite state machine diagram* contains one or more nested state machine diagrams.

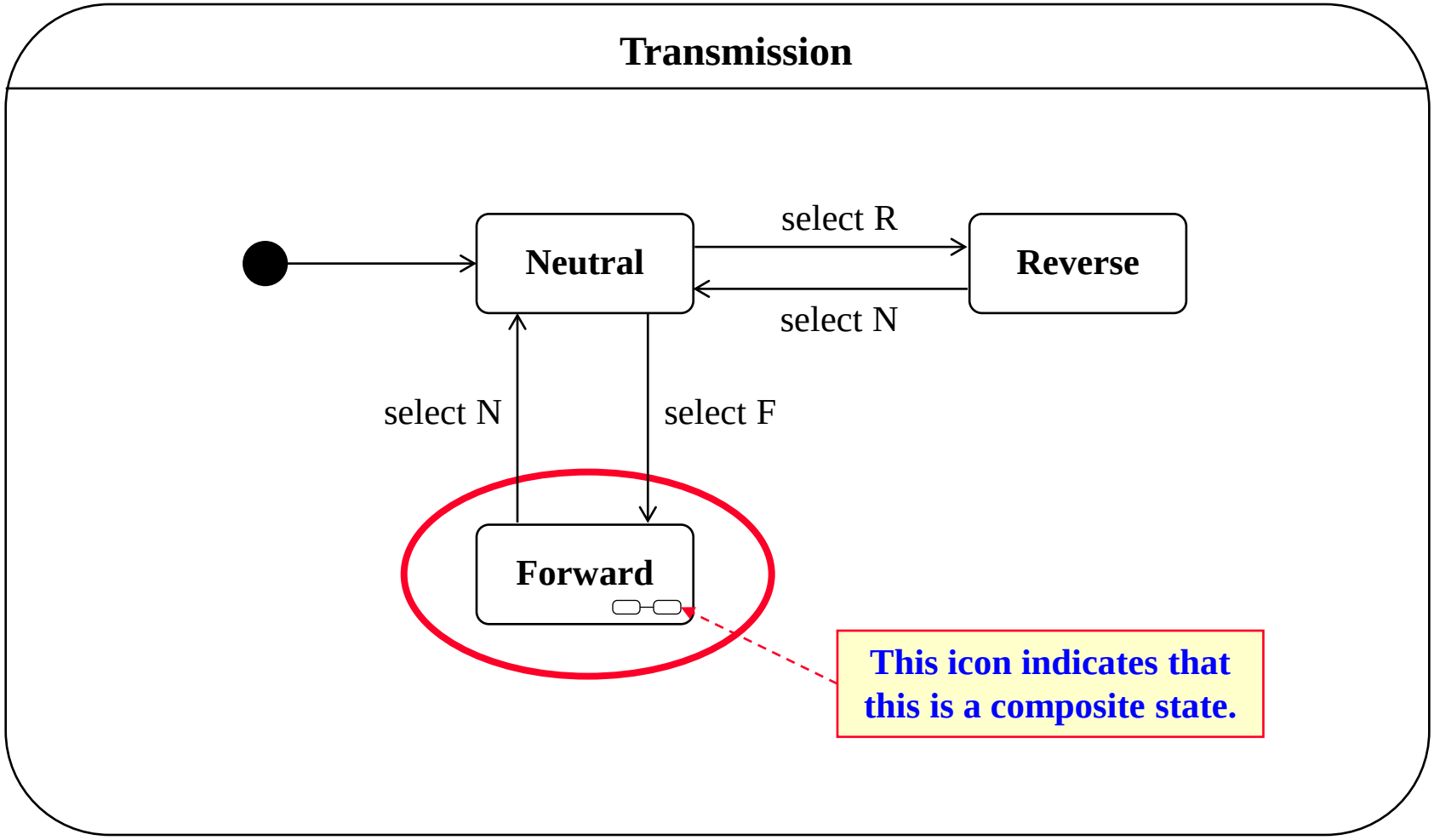
sequential composite state machine diagram

- An object is in exactly one state in one of the (nested) state machine diagrams.
  - ☞ This corresponds to an or-condition on all state machine diagrams.
- It is used to abstract/generalize states.

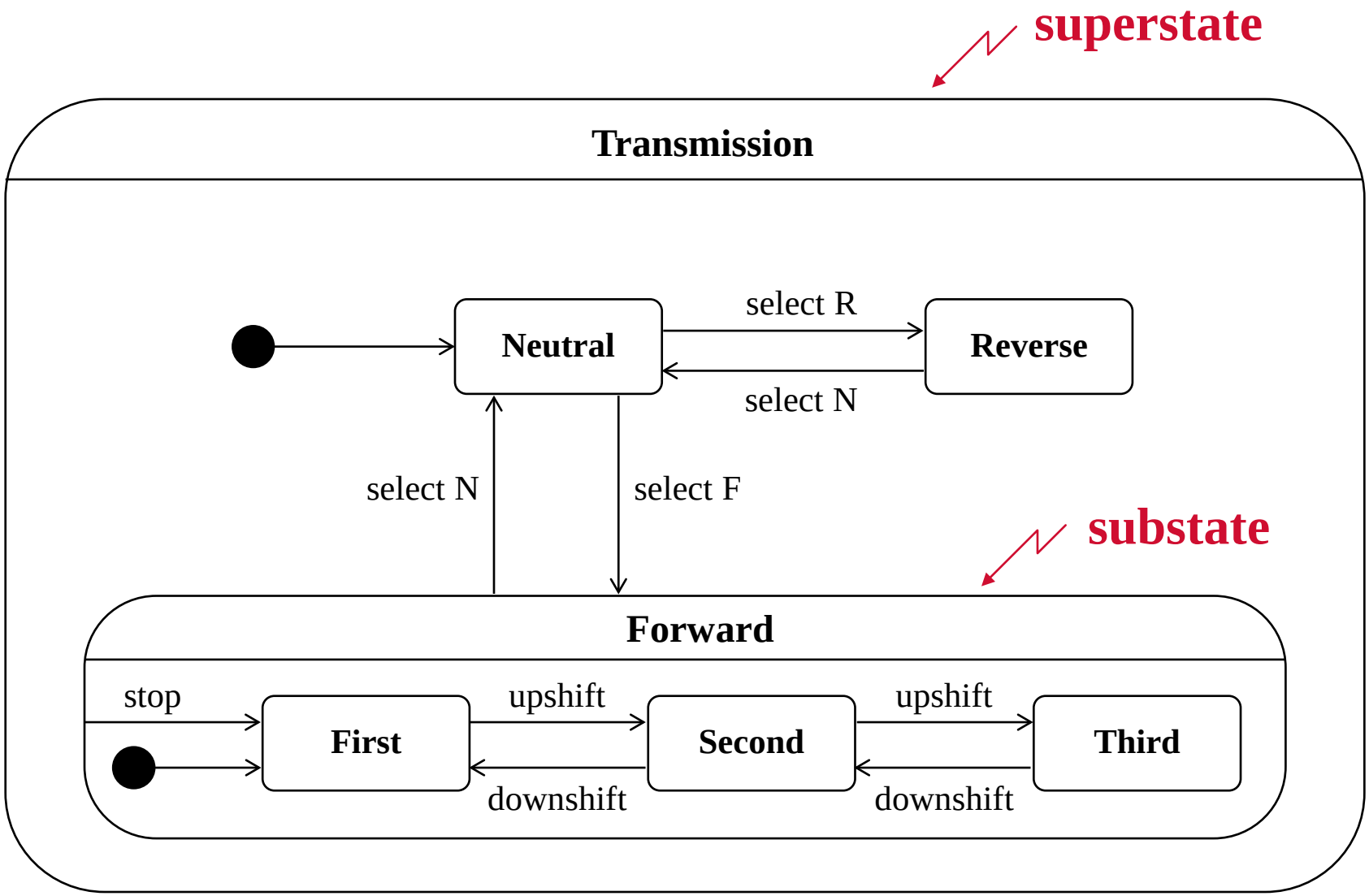
concurrent composite state machine diagram

- An object is in exactly one state in each of the regions of the state machine diagrams.
  - ☞ This corresponds to an and-condition on all regions.
- It is used to show multi-threading behavior.

SEQUENTIAL COMPOSITE STATE MACHINE DIAGRAM

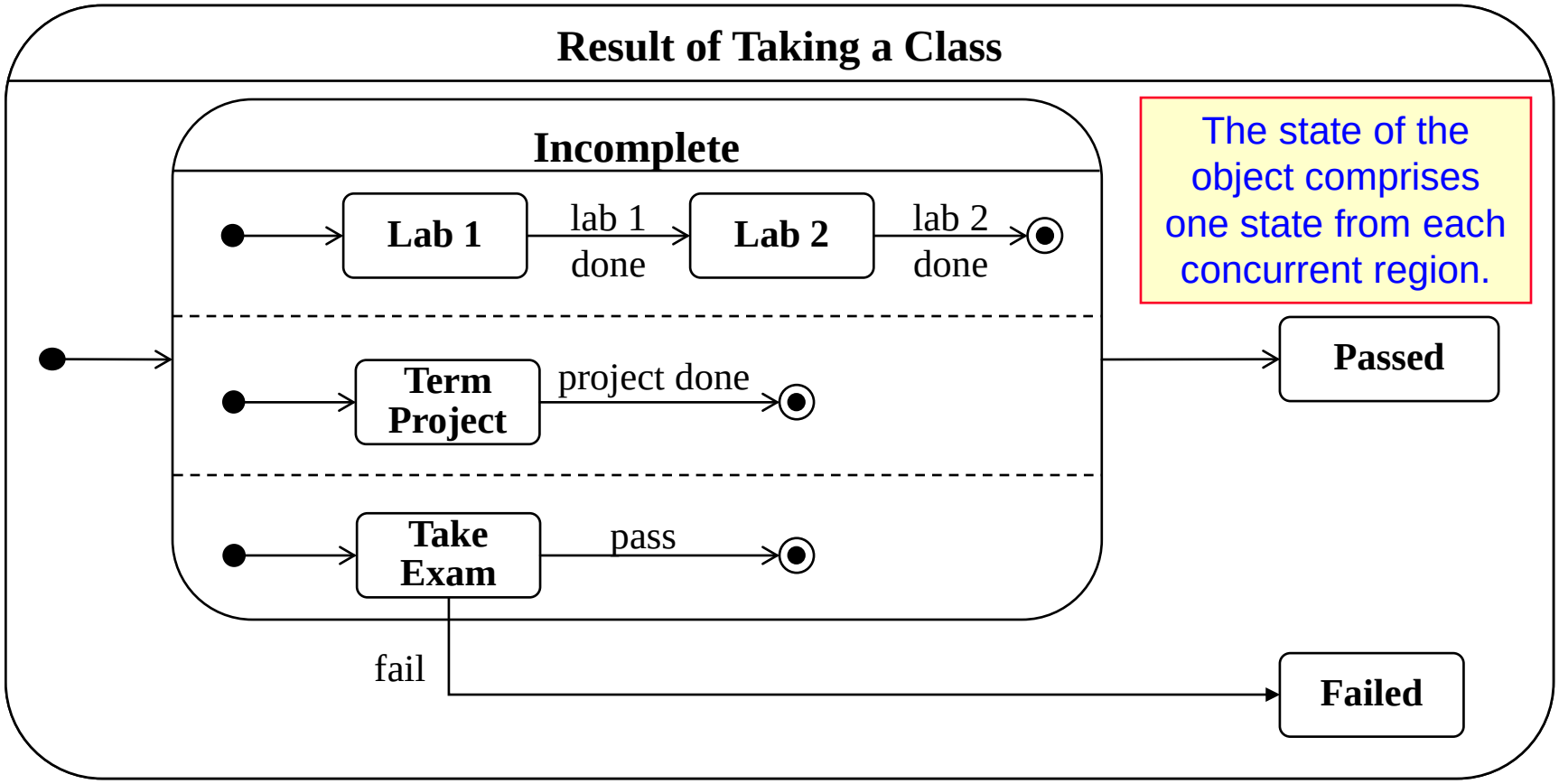


SEQUENTIAL COMPOSITE STATE MACHINE DIAGRAM



CONCURRENT COMPOSITE STATE MACHINE DIAGRAM

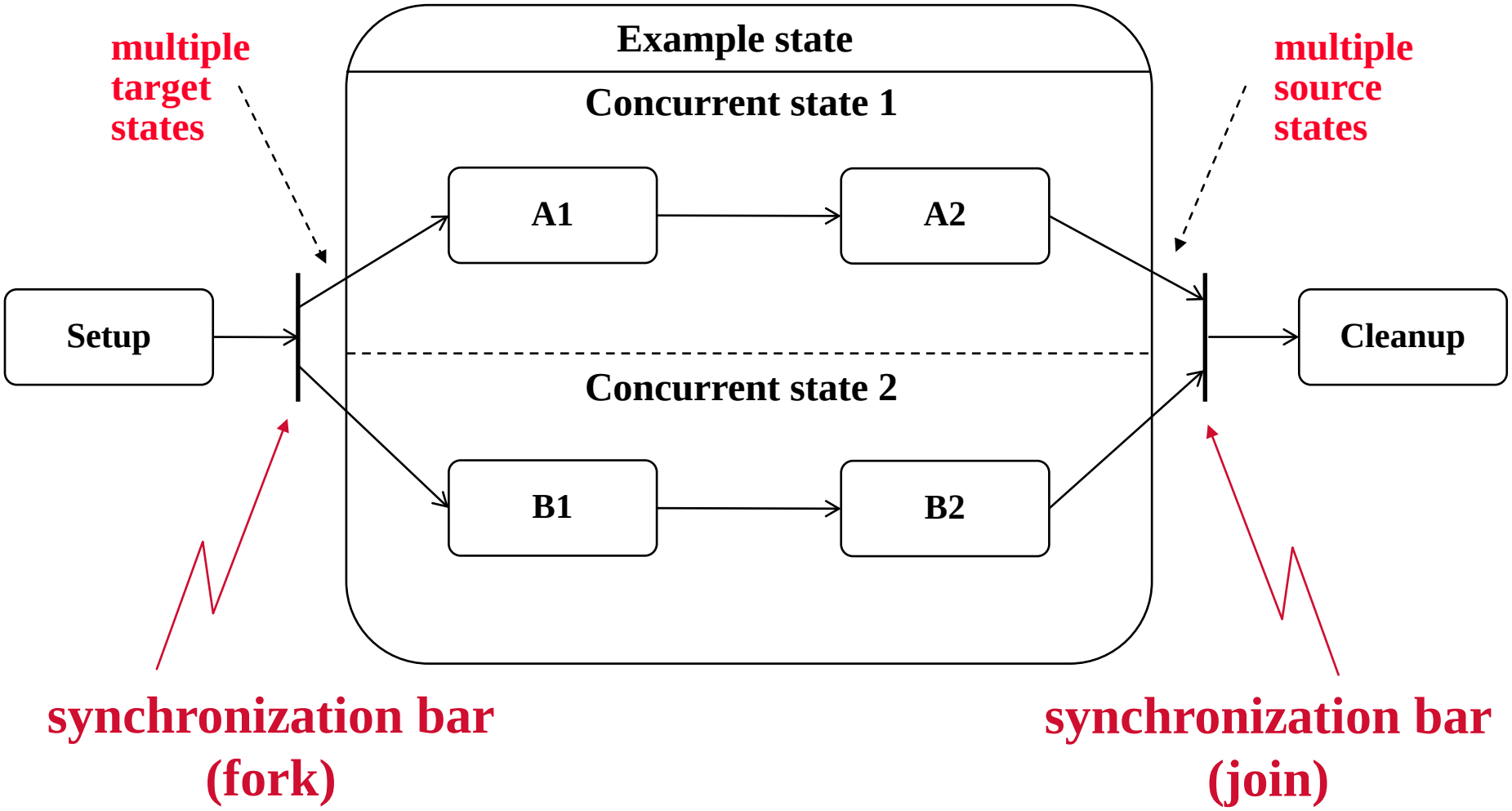
- Concurrency arises when an object can be partitioned into sub-sets of attributes or links, each with its own state machine diagram.



COMPOSITE STATE MACHINE DIAGRAM: TRANSITIONS

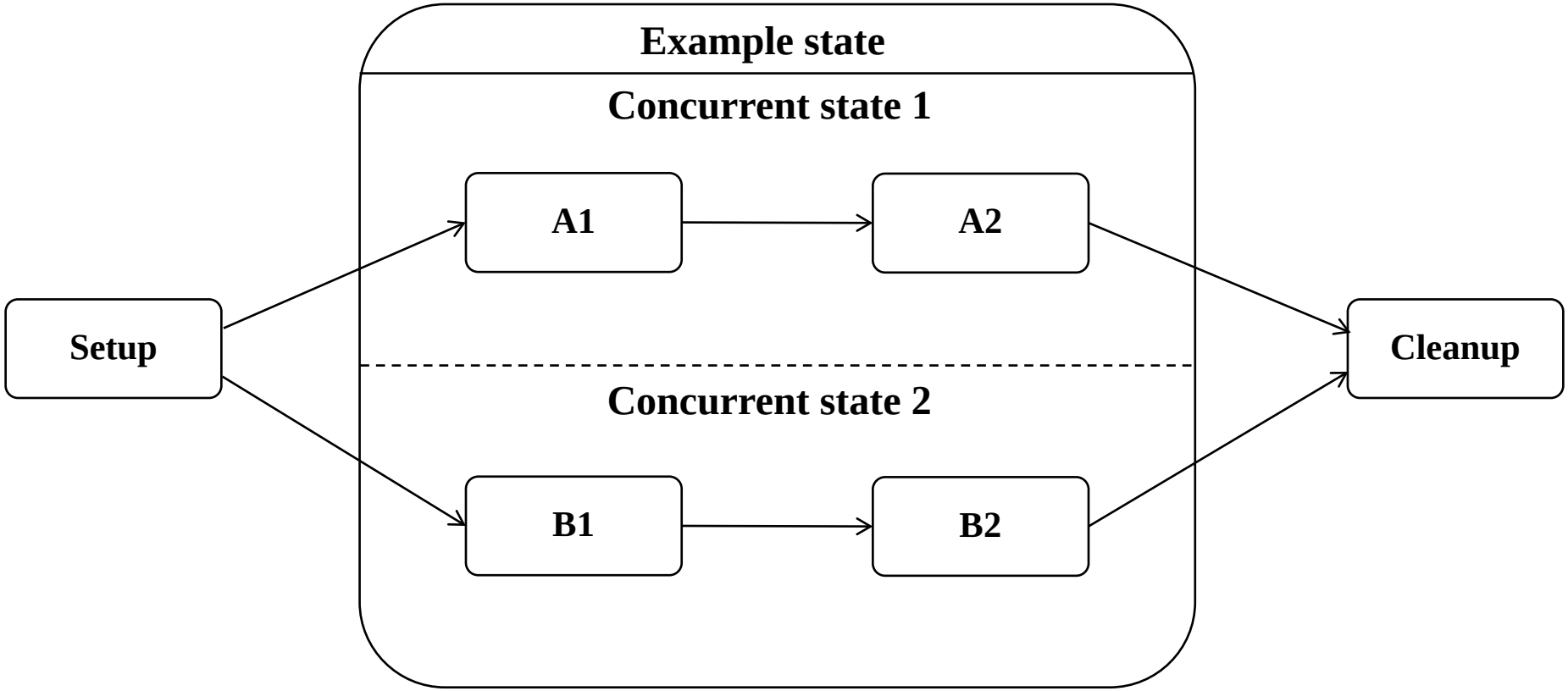
- A transition to the boundary ≡ a transition to the initial state.
  - ☞ **The entry activity of all regions entered are performed.**
- There may be transitions directly into a composite state region.
- A transition from the boundary ≡ a transition from the composite state.
  - ☞ **The exit activity of all regions exited are performed.**
- There may be transitions directly from within a composite state region to an outside state.
- A transition may have multiple source and target states.
  - ☞ **This represents a synchronization or splitting of control.**

CONCURRENT COMPOSITE STATE MACHINE DIAGRAM: SYNCHRONIZATION



CONCURRENT COMPOSITE STATE MACHINE DIAGRAM: SYNCHRONIZATION (cont'd)

Is there a difference in execution between this state machine diagram and the previous one?



WHEN TO USE A STATE MACHINE DIAGRAM

- A state machine diagram is good at describing the behavior of an object across several use cases.

☞ **It is not necessary to produce a state machine diagram for every class.**

- A state machine diagram should be used only for classes with **significant** dynamic behaviour.

SYSTEM ANALYSIS & DESIGN

STATE MACHINE DIAGRAM EXERCISE